

CLEANROOMX

Complete Solution Manual & User Guide

Design • Simulation • HVAC • Networks • BIM/IFC • Verification • Evidence •
ProofGraph

Repository state verified on 30 September 2026

Stable release: v0.102.1 | Current main: ae80aec3f61f... | Latest merged PR: #594

PR #595: open, mergeable, exact head e2fd382..., CI run #1928 SUCCESS

Engineering screening and auditable numerical evidence — not a substitute for
certification, CFD, commissioning/TAB, manufacturer approval, or regulatory
acceptance.

Document Control & Verified Repository Snapshot

Item	Verified value
---	---
Manual date	30 September 2026
Repository	3more102/CleanroomX-Cleanroom-Design-Simulation-Verification-Platform
Default branch	main
Current main SHA	ae80aec3f61f009597a1b01f05d67fb91b545d70
Current main latest merged PR	#594 - Preserve full precision in damper flow redistribution
Stable release	v0.102.1
Package version on current main	0.102.1
Supported Python	3.11 / 3.12 / 3.13
Stable release validation	1008 passing tests on each supported Python version at PR #487 release gate
Project schema	cleanroomx.project, schema version 1
ProofGraph schema	cleanroomx.proofgraph, schema version 1 (current main / unreleased)
Open PR at cutoff	#595 - Use full-precision standalone fan roots in HVAC consistency; CI SUCCESS; not yet merged

Interpretation of status labels Stable means documented in the v0.102.1 release closure. Current main / unreleased means implemented on main after the stable release. Pending means the code is in an open PR and must not be treated as part of main.

This manual is organized around operator tasks and engineering reasoning rather than repository chronology. Every current-main-only capability is called out where it matters. The full repository CHANGELOG remains the authoritative commit-by-commit history.

Contents

- 01 Product scope and architecture
- 02 Installation and first launch
- 03 Project model and files
- 04 Desktop GUI workflow
- 05 2D/3D spatial design
- 06 Spatial-engineering synchronization
- 07 Core room/project verification
- 08 Particle decay and recovery
- 09 Psychrometrics, thermal and HVAC
- 10 Duct, branch and loop airflow networks
- 11 Room pressure / leakage network [current main]
- 12 Fan/system studies and fan-network composition

- 13 Uncertainty, damper scenarios and numerical precision
- 14 Design foundation and consistency [current main]
- 15 Compliance, design assurance and snapshots [current main]
- 16 ProofGraph evidence model [current main]
- 17 BIM/IFC interoperability [current main enhancements]
- 18 Persistence, recovery, revisions and run history
- 19 Reporting, bundles, batch automation and diagnostics
- 20 Command-line reference
- 21 Worked solution examples
- 22 Troubleshooting and failure handling
- 23 Security, deployment, migration and rollback
- 24 Validation evidence and current development status
- 25 Engineering limits, glossary and source map

01 PRODUCT SCOPE AND ARCHITECTURE

What CleanroomX is, what it owns, and where the engineering boundary remains external.

CleanroomX is a Python engineering platform for cleanroom design, simulation, verification, spatial planning, HVAC analysis, fan/network studies, uncertainty, BIM interoperability, reporting, and auditable numerical/provenance evidence.

Its central architectural rule is that the desktop application and command-line tools are shells over shared backend services. Engineering equations are not reimplemented inside GUI callbacks. This keeps GUI, batch, CLI, report, and assurance results aligned to one canonical calculation path.

Figure 1. CleanroomX consolidated architecture.

Core capability families

Family	Capabilities
---	---
Spatial design	Synchronized 2D/3D rooms, elevations, classifications, openings, devices, pressure visualization, validation, deterministic edit history.
Verification	Room/project ACH, particles, pressure cascade, recovery qualification and explicit aggregate completeness.
HVAC / thermal	Psychrometrics, sensible/latent/makeup loads, airflow balance, preliminary fan duty, bounded thermal uncertainty.
Networks	Duct paths, fixed-demand branches, looped quadratic networks, variable friction, room pressure/leakage network.
Fan studies	Supplied fan curve operating point, speed/affinity studies, passive-network and loop integrations, uncertainty.
Assurance	Consistency, dossiers, compliance rule packs, design assurance, ProofGraph, replay/integrity evidence.
Project lifecycle	Strict JSON, guarded atomic saves, autosave/recovery, revisions, undo/redo, run history, stale-evidence detection.
Handoff / automation	Portable project bundles, portable HTML reports, deterministic project batch execution, diagnostics, CLI workflows.

Non-negotiable engineering boundary

Not certification CleanroomX provides screening, simulation, verification, and software/provenance evidence. It does not by itself establish ISO cleanroom certification, CFD validation, commissioning/TAB acceptance, manufacturer approval, physical/statistical uncertainty, or regulatory compliance.

Project-specific requirements, acceptance limits, leakage coefficients, manufacturer performance data, weather/design data, measurement uncertainty, and regulatory criteria remain explicit operator inputs. CleanroomX is designed to make those assumptions visible rather than invent them.

02 INSTALLATION AND FIRST LAUNCH

Windows repository checkout, Python installation, readiness checks, demo startup, and optional BIM support.

Windows repository checkout

```
cd C:\CleanroomX
.\start-cleanroomx.ps1

# Command Prompt / Explorer-compatible launcher
start-cleanroomx.cmd

# Headless readiness check
.\start-cleanroomx.ps1 --check
```

The PowerShell launcher prefers .venv\Scripts\python.exe, otherwise uses the Python launcher or python command. It sets the repository src directory on PYTHONPATH and defaults to --demo when no arguments are supplied.

Normal Python installation

```
python -m venv .venv
python -m pip install -e .
cleanroomx-gui --check
cleanroomx-gui --demo
```

CleanroomX requires Python 3.11 or newer within the supported 3.11/3.12/3.13 matrix. The base package declares no mandatory third-party runtime dependencies. Development adds pytest; native IFC reading is optional.

Development / validation install

```
python -m pip install -e .[dev]
python -m pytest -q
```

Native IFC support

```
pip install "cleanroomx[bim]"
```

If the GUI does not open Confirm that the Python installation includes Tk support. On Linux CI-style systems install Tk and Xvfb; on Windows use the repository launcher to avoid PATH ambiguity.

First five minutes

1. Run cleanroomx-gui --check. Do not proceed if the registry readiness check fails.
1. Open cleanroomx-gui --demo to inspect a complete project with rooms, HVAC, fan/network and evidence workflows.
1. Open Design 2D + 3D and verify that Process, Preparation, and Ante/Airlock appear with pressure and devices.
1. Select an analysis, choose Validate, then Run. Review result JSON, diagnostics/provenance, and Markdown report.
1. Export a portable HTML report or project bundle only after the result is fresh.

Figure 2. Recommended operating sequence.

03 PROJECT MODEL AND FILES

Schema-v1 projects, analyses, strict JSON, path context, migration, and input identity.

Project document

Desktop projects use schema cleanroomx.project, schema version 1. The document stores project metadata, an ordered list of analyses, an optional active analysis identifier, and spatial layout metadata under project.metadata.spatial layout.

```
{ "schema": "cleanroomx.project", "schema_version": 1, "application_version": "0.102.1", "project":  
  { "name": "My Cleanroom", "metadata": {} }, "analyses": [ { "id": "verification", "name": "Facility  
verification", "kind": "project_verification", "input": {...} } ], "active_analysis_id": "verification" }
```

Strict JSON boundary

- Malformed JSON is rejected.
- Duplicate object keys are rejected at strict engineering ingestion boundaries.
- NaN, Infinity and -Infinity are rejected; outputs use finite strict JSON.
- Unsupported future schemas, duplicate analysis IDs, invalid active-analysis references, and unsupported kinds fail closed.
- Normal project JSON is capped at 64 MiB on current main; the same ceiling is used for saves and bundle project members.

Additive-field preservation

Within schema version 1, unknown additive strict-JSON fields at the document, project block, and analysis-record levels are preserved across open/edit/save cycles. CleanroomX-owned fields remain authoritative when an extension attempts to reuse a reserved key.

Legacy migration behavior

Two legacy single-analysis shapes can be migrated into schema-v1 in memory. A migrated legacy file opens as an unsaved converted copy. Save Project routes to Save Project As, and the first new save cannot overwrite the original legacy source. CleanroomX intentionally does not synthesize reverse migrations.

File-backed analyses and path context

Consistency and dossier workflows may reference external engineering files. Relative paths resolve against the saved project directory. Save Project As rebases relative references when the destination directory changes. During a run, referenced files are fingerprinted before and after execution; changed or unstable dependencies invalidate the run instead of publishing mixed-revision evidence.

04 DESKTOP GUI WORKFLOW

How to create, validate, run, inspect, and export analyses safely.

Normal operator sequence

1. Create a new project or open a .cleanroomx.json project.
1. Add an analysis from the application catalog, or select an existing analysis.
1. Edit or import the strict JSON object in the input editor.
1. Choose Validate to run the real backend parser/validation path without executing the engineering analysis.
1. Choose Run. CleanroomX executes the backend in a worker thread while the GUI remains responsive.
1. Review normalized result JSON, diagnostics/provenance, Markdown report, and any available plot.
1. Export input, result, run bundle, Markdown, portable HTML report, or save the project only after confirming freshness.

Run ownership and the Abandon action

Abandon suppresses the pending result but does not force-terminate the Python worker. The application keeps execution exclusive and locks conflicting input mutation until that worker exits, preventing overlapping backend runs.

Freshness guard

Rerun after input changes Completed results are bound to the canonical SHA-256 of the exact submitted analysis input. Before cache restore, worker acceptance, and exports, CleanroomX checks that the current kind/input still match. A mismatch clears stale evidence and requires a rerun.

Headless application contract check

```
cleanroomx-gui --check
```

This validates unique workflow keys, parser/runner contracts, custom adapters, and callable target bindings without opening a window. Normal GUI startup performs the same registry validation before creating the Tk root.

05 2D / 3D SPATIAL DESIGN

Canonical geometry, editing controls, room/device semantics, visualization, and validation.

The 2D editor and 3D viewer share one canonical spatial model. There is no separate 3D geometry copy. Existing projects without spatial metadata still open normally.

Spatial model

Object	Stored information
---	---
Floor	stable ID, name, elevation, default ceiling height, metric units
Room	stable ID, X/Y, length, width, height, floor elevation, optional pressure, classification, analysis-room link
Device/opening	stable ID, type, room assignment, X/Y/Z, dimensions, orientation; wall openings can include wall side and swing
View	2D/3D zoom and pan, azimuth/elevation, snap and visibility flags
Sync baseline	analysis identity, room mapping identity and last synchronized dimensions

2D editing

- Create, select, drag, resize, duplicate, delete, and edit rooms.
- Place doors, windows, generic openings, supply, return, exhaust, FFUs, equipment, sensors, and transfer openings.
- Configure metric grid spacing; toggle snap-to-grid, labels, devices, pressure and pressure-cascade relationships independently.
- Use zoom, pan, fit-to-view and explicit 2D reset without altering model geometry.
- Room drag moves assigned devices. Inspector X/Y changes also translate assigned devices by the same offset.
- Room/device duplication receives new stable IDs and is one undo/redo transaction. Room copies deliberately do not copy pressure, engineering link, or synchronization baseline.

3D viewer

The pure-Tk 3D view projects the same room footprints, elevations and devices. Mouse wheel zooms; middle/right drag pans; Shift+left-drag orbits. Fit derives zoom and both pan axes from all room floor/ceiling corners, so elevated or large layouts are actually recentered.

Spatial integrity checks

- non-finite or non-positive dimensions;
- duplicate stable IDs and duplicate room names;
- overlapping room footprints;
- dangling room references and orphan/unassigned devices;
- devices outside assigned rooms or above valid elevation;
- unsupported device types or malformed view flags.

Geometry checks are not certification checks A spatial validation pass means the internal model is geometrically/self-consistently represented. It does not establish ISO/GMP compliance, airflow performance, CFD validity, or commissioning acceptance.

06 SPATIAL-ENGINEERING SYNCHRONIZATION

Explicit, dimension-only exchange with deterministic synchronization states.

Figure 3. CleanroomX synchronization boundary.

Push and Pull

Use Push to analysis when spatial geometry is authoritative. Use Pull from analysis when engineering room geometry is authoritative. For room-verification and project-verification workflows, both directions synchronize only length, width, and height.

- Pull preserves spatial X/Y placement.
- Observed pressure and other engineering evidence are not synchronized into geometry.
- Fresh completed verification pressure may be projected into 2D/3D for visualization without being persisted as spatial evidence.
- All room mappings and dimensions are preflighted before mutation; an invalid later room cannot leave a partial update.
- After a real engineering-input change, the affected analysis must be validated/run again.

Synchronization states

State	Meaning
---	---
synchronized	Mapped room dimensions match the persisted synchronization baseline.
geometry newer	Persisted baseline proves spatial dimensions changed since last sync.
engineering newer	Persisted baseline proves engineering dimensions changed since last sync.
conflicting	Both sides differ, or provenance is insufficient to prove which side is newer.
unmapped	No valid explicit room mapping exists.

07 CORE ROOM AND PROJECT VERIFICATION

Volume, ACH, particle criteria, observed pressure, pressure-cascade checks, and aggregate completeness.

Room volume and ACH

Room volume

$$V = \text{length} * \text{width} * \text{height}$$

Dimensions are in metres; V is in cubic metres.

Nominal supply air changes per hour

$$\text{ACH} = \text{supply_airflow_m3_h} / V$$

The implemented calculation uses supply airflow divided by geometric room volume.

Project verification

A project can contain multiple rooms plus explicit pressure-cascade relationships. Room results and pressure relationships are evaluated from supplied engineering data; CleanroomX does not invent a missing room pressure.

```
"pressure_cascade": [ {"higher_pressure_room": "Process", "lower_pressure_room": "Preparation",  
  "min_delta_pa": 10} ]
```

Aggregate verification states

Status	Meaning
---	---
fail	At least one evaluated finding fails.
pass_with_unchecked	No evaluated finding fails, but at least one configured criterion remains unchecked.
pass	Every included finding is evaluated and passing.
not_checked	Nothing in the aggregate was evaluated.

The historical passed boolean remains a backward-compatible no-failure predicate. Use status plus complete when deciding whether evidence is actually complete.

08 PARTICLE DECAY AND RECOVERY

First-order screening model, CLI use, recovery qualification, and limitations.

Well-mixed decay screening

Particle concentration decay

$$C(t) = C_0 * \exp(-(ACH/60) * \eta * t)$$

C_0 and $C(t)$ are concentration per m³, t is in minutes, ACH is 1/h, and η is an effective single-pass removal fraction.

Recovery time

Time to target

$$t = \ln(C_0 / C_{\text{target}}) / ((ACH/60) * \eta)$$

If the target is already greater than or equal to the initial concentration, the implemented function returns 0 minutes.

Main CLI

```
cleanroomx decay --initial 1000000 --ach 30 --minutes 10 --efficiency 1.0
cleanroomx recovery --initial 1000000 --target 100000 --ach 30 --efficiency 1.0
```

Observed recovery qualification

The dedicated recovery-test workflow can evaluate measured time-series samples against a target concentration and optional maximum recovery time. Instrument identity, sample location, occupancy state and provenance can remain visible in the result/report.

Model limitation The simple decay equation is not CFD and does not model active particle sources, deposition, leakage, imperfect mixing, local recirculation, or room-scale velocity fields.

09 PSYCHROMETRICS, THERMAL AND HVAC

Air-state properties, thermal loads, airflow balance, preliminary fan duty and bounded endpoint uncertainty.

Steady room airflow balance

Net surplus

$$\text{net_surplus} = \text{supply} + \text{transfer_in} - \text{return} - \text{exhaust} - \text{transfer_out}$$

A positive surplus is airflow available for exfiltration or another unmodeled outflow path. A negative value means infiltration or another unmodeled inflow is needed to close the balance.

Surplus margin

$$\text{surplus_margin} = \text{net_surplus} - \text{minimum_required_surplus}$$

Balance decisions use full-precision airflow state; presentation values are rounded only after the engineering calculation.

Preliminary supply-fan duty

Total static pressure

$$P_{\text{static}} = P_{\text{duct}} + P_{\text{coil}} + P_{\text{terminal_filter}} + P_{\text{other}}$$

All terms are explicit project inputs except duct loss when a computed critical path is supplied.

Fluid, shaft and electrical power

$$\begin{aligned} Q_s &= \text{airflow_m3_h} / 3600 \\ \text{air_power_W} &= Q_s * P_{\text{static}} \\ \text{shaft_power_W} &= \text{air_power_W} / \text{fan_efficiency} \\ \text{electrical_input_W} &= \text{shaft_power_W} / \text{motor_efficiency} \end{aligned}$$

This is preliminary steady-state sizing. System effect, dirty-filter allowance, VFD/control losses, altitude correction and final manufacturer selection are not inferred.

Thermal uncertainty [bounded deterministic screening]

The thermal uncertainty workflow enumerates endpoint combinations for uncertain room/outdoor dry-bulb temperature, relative humidity and pressure, then reports conservative intervals for humidity ratio, enthalpy, specific volume and dew point. Load and airflow intervals are propagated without assuming monotonicity of every derived property.

Cooling/heating capacity requirement

$$Q_{\text{cool}} = \max(Q_{\text{net}}, 0) * \text{margin_multiplier} \quad Q_{\text{heat}} = \max(-Q_{\text{net}}, 0) * \text{margin_multiplier}$$

Capacity status is pass when available capacity covers the whole requirement interval, fail when the whole interval exceeds capacity, indeterminate when the available capacity lies inside the interval, and not_checked when no available capacity is configured.

```
cleanroomx-hvac examples/hvac_demo.json
cleanroomx-thermal-uncertainty examples/thermal_uncertainty_demo.json --format json
cleanroomx-psychrometric-uncertainty examples/psychrometric_uncertainty_demo.json
```

10 DUCT, BRANCH AND LOOP AIRFLOW NETWORKS

Darcy-Weisbach paths, fixed-demand branching, nonlinear loops, variable friction and damper scenarios.

Duct section equations

Area and hydraulic diameter

circular: $A = \pi D^2/4$, $D_h = D$ rectangular: $A = \text{width} \times \text{height}$, $D_h = 2 \times \text{width} \times \text{height} / (\text{width} + \text{height})$

A section uses either circular diameter or both width and height.

Velocity pressure and losses

$v = Q/A$ $P_v = 0.5 \times \rho \times v^2$ $P_{\text{friction}} = f \times (L/D_h) \times P_v$ $P_{\text{local}} = K \times P_v$ $P_{\text{total}} = P_{\text{friction}} + P_{\text{local}}$

Darcy factor may be supplied directly or calculated from explicit roughness and kinematic viscosity. Full-precision calculation state is retained for downstream composition.

Looped network model

Fixed quadratic edge law

$\Delta P = R \times Q \times \text{abs}(Q)$

Q is signed m³/s. Negative solved flow is valid; input direction is only the positive sign convention.

Geometry-derived resistance

$R = 0.5 \times \rho \times (f \times L/D_h + K) / A^2$

A fixed geometry-derived resistance may use a supplied Darcy factor or automatic friction resolved at an explicit reference airflow.

Loop solver

CleanroomX validates a connected graph and balanced node injections, constructs a spanning-tree pressure estimate, solves node continuity using damped Newton iteration, and reports node mass-balance and edge pressure-law residuals. Non-convergence is an error, not a false solved result.

Variable-friction loop

Automatic-friction geometry edges can be iterated: solve the loop, recompute Reynolds number and Darcy friction from the absolute solved airflow, rebuild target resistance, optionally relax, and repeat until resistance closure. Directly supplied resistances and user-supplied Darcy factors remain fixed.

Damper scenario study

Case resistance

$R_{\text{case}} = \text{multiplier} \times R_{\text{base}}$

The multiplier must be ≥ 1.0 . CleanroomX does not convert actuator command or blade angle into resistance.

```
cleanroomx-duct-flow examples/duct_network_demo.json
cleanroomx-loop-flow examples/looped_network_demo.json
cleanroomx-loop-friction examples/variable_friction_loop_demo.json
cleanroomx-damper-study examples/damper_study_demo.json --format json
```

11 ROOM PRESSURE / LEAKAGE NETWORK

[CURRENT MAIN]

Steady-state multizone pressure solution with explicit mechanical airflows and user-parameterized leakage/opening laws.

Release status This first-class pressure/leakage network was added on current main after the v0.102.1 stable release. It is implemented and documented on main at the manual cutoff.

Nodes and mechanical injection

Mechanical injection

```
mechanical_injection = supply - return - exhaust
```

At least one fixed-pressure node is required; every unknown-pressure node must be connected to a fixed boundary through configured paths.

Power-law path

Power-law leakage/opening flow

$$Q = C * \text{sign}(dP) * |dP|^n$$

C is user supplied in m³/s/Paⁿ. CleanroomX accepts n from 0.5 to 1.0.

Orifice path

Orifice flow

$$Q = C_d * A * \text{sign}(dP) * \sqrt{2 * |dP| / \rho}$$

C_d, opening area A and air density rho are explicit user inputs.

Near-zero regularization and offsets

Each path has linearization_pressure_pa. Below that pressure magnitude, CleanroomX uses a linear continuation matched to the configured nonlinear law, avoiding the derivative singularity near zero. pressure_offset_pa may be added explicitly to the node-pressure difference, but wind and stack pressure are not calculated automatically.

Solver and evidence

- Damped Newton simultaneous solution of unknown room pressures.
- Reported iteration count, iteration limit, mass-balance tolerance and maximum unknown-node residual.
- Every solved node pressure, path airflow/direction and local dQ/dP sensitivity.
- Dominant pressure path for each node and fixed-boundary balancing flow.
- Explicit pressure-target margins; target failure yields solved_with_target_violations rather than hiding the physical solution.
- Non-convergence or non-finite derived state fails explicitly.

```
cleanroomx-pressure-network examples/pressure_network_demo.json
cleanroomx-pressure-network examples/pressure_network_demo.json --format json
```

12 FAN/SYSTEM STUDIES AND FAN-NETWORK COMPOSITION

Bounded fan-curve interpolation, operating points, speed laws, passive networks and nonlinear loop integration.

Standalone fan/system operating point

Quadratic system curve

$$\text{deltaP_system} = \text{deltaP_fixed} + R * Q^2$$

Fan pressure is piecewise-linearly interpolated only between supplied fan-curve points. CleanroomX does not extrapolate outside the supplied airflow range.

Within each supplied fan-curve segment, CleanroomX finds crossings between interpolated fan pressure and system pressure. A solved result reports airflow, fan pressure, system pressure, residual, fluid air power $Q \cdot \text{deltaP}$, and the fan segment. If no crossing exists in supplied data, the status is `no_intersection_in_supplied_range`.

```
cleanroomx-fan-curve examples/fan_operating_point_demo.json
cleanroomx-fan-speed examples/fan_speed_study_demo.json
cleanroomx-fan-network examples/fan_network_demo.json
cleanroomx-fan-loop examples/fan_loop_network_demo.json
```

Speed / affinity studies

Speed workflows scale supplied reference fan points using explicit speed ratios: airflow scales with speed ratio, pressure with speed ratio squared. Cubic scaling is reported as an affinity-law homologous power indicator, not as actual electrical input power unless an explicit separate efficiency model supports that interpretation.

Fan + network integration

Fan-network and fan-loop workflows couple bounded fan-curve root solving to passive network or loop-network calculations. Recent current-main hardening keeps the operating root and downstream composition at full precision, using public rounded values only for presentation.

No extrapolation Fan-curve workflows deliberately do not extrapolate manufacturer data. Outside-supplied-range is an engineering result, not an invitation to extend the curve.

13 UNCERTAINTY, DAMPER SCENARIOS AND NUMERICAL PRECISION

How CleanroomX treats bounded uncertainty and why rounded display values never become engineering inputs.

Deterministic bounded uncertainty

CleanroomX uncertainty workflows use explicitly supplied bounds and deterministic corner/endpoint enumeration. They are not statistical confidence intervals unless the underlying project provides such a model. Missing provenance and incomplete corners remain visible rather than being promoted to complete evidence.

Presentation boundary rule

Full precision inside; rounding only at the edge Current main has systematically hardened HVAC, duct, branch, pressure, fan/network, variable-friction, damper and thermal/ProofGraph paths so downstream decisions use canonical full-precision calculation state rather than rounded public result fields.

Current-main hardening (30 Sep 2026)	Effect
---	---
Pressure-design verdicts	Use full-precision pressure-network node state for fine-tolerance comparisons.
Variable-friction feedback	Use exact solved edge airflow for thresholding, Reynolds/friction updates and closure.
Fan-network composition	Use full-precision operating root for redistribution and pressure checks.
Fan-loop composition	Use full-precision fan root for loop solve and residual.
Fan-loop uncertainty	Build envelopes from canonical corner calculations.
HVAC/fan dossier consistency	Recompute canonical governing airflow instead of comparing rounded HVAC totals.
Thermal ProofGraph evidence	Store canonical full-precision interval evidence that actually drove capacity status.
Damper redistribution	Compute delta and percent change from canonical baseline/case state, not rounded edge tables.

Pending PR #595

PR #595 is open, mergeable, and its CI run #1928 succeeded at the cutoff. It changes dossier-owned standalone fan/HVAC airflow consistency to compare against the canonical full-precision standalone fan root instead of the public three-decimal fan airflow. Because the PR was not merged, this behavior is not included in current main in this manual.

14 DESIGN FOUNDATION AND CONSISTENCY

[CURRENT MAIN]

Explicit requirements, preliminary air-system design, and read-only consistency composition without hidden standards assumptions.

Release status The Phase 1 design foundation was added after v0.102.1 and is current-main/unreleased functionality at the cutoff.

Design requirements

The design_requirements workflow turns explicit room requirements and selected reference profiles into traceable targets. Derived evidence currently includes room area, volume, ACH-based airflow and explicitly entered sensible-load totals while retaining classification, intended process, occupancy, temperature/RH range, pressure target, recovery target, filtration requirement, contamination assumptions, strategy and operating mode.

Preliminary air-system design

Air-balance sizing driver

$$\max(0, \text{exhaust} + \text{transfer_out} + \text{minimum_surplus} - \text{transfer_in})$$

This is one candidate driver alongside minimum ACH, sensible-load airflow and minimum outdoor airflow. The largest evaluated driver becomes the governing preliminary supply airflow.

The workflow can propose return/makeup balance and estimate capacity-based counts for user-supplied FFUs/filter units and supply/return/exhaust terminals. Strategy labels do not inject hidden airflow requirements.

Design consistency

design_consistency composes the canonical design-requirements and air-system services. It compares only quantities explicitly represented on both sides: room set, dimensions, minimum ACH, ACH-derived supply airflow, entered sensible load, and air-system room temperature against the configured requirement range.

Every numeric comparison uses an explicit absolute tolerance. Missing requirement evidence remains not_checked. Pressure, RH, recovery, filtration, contamination and free-text strategy are deliberately outside this cross-check.

Pressure design consistency

The separate pressure_design_consistency workflow requires explicit room -> node/reference-node mappings. It compares configured pressure_target_pa only with solved pressure(node) - pressure(reference_node) using a caller-supplied absolute tolerance. The sign is preserved and no reference room is inferred.

```
cleanroomx-project-run project.cleanroomx.json # In a project, add analysis kinds such as: #
design_requirements # air_system_design # design_consistency # pressure_design_consistency
```

15 COMPLIANCE, DESIGN ASSURANCE AND SNAPSHOTS [CURRENT MAIN]

Versioned rule-pack checks, matrix aggregation, traceability digests and deterministic replay artifacts.

Design assurance matrix

design_assurance composes one canonical design_consistency study, an optional pressure_design_consistency study, and one or more versioned compliance_check rule-pack evaluations. At least one compliance check is required.

Aggregate	Rule
---	---
fail	Any included component records a failure.
not_checked	Every component is entirely not checked.
pass_with_unchecked	No component fails, but unresolved evidence remains.
pass	Every component is complete and passes.

Traceability

Compliance components retain rule-pack identity/version/source, rule-pack SHA-256, exact evidence-document SHA-256 and normalized component-result digest. Design and pressure-consistency components retain their full canonical results and result digests. traceability_sha256 deterministically binds the component traceability manifest.

Design assurance snapshots

```
cleanroomx-assurance-snapshot create examples/design_assurance_demo.json design_assurance.snapshot.json
cleanroomx-assurance-snapshot verify design_assurance.snapshot.json
```

- Schema v1: cleanroomx.design-assurance-snapshot.
- Freezes exact UTF-8 source bytes, source SHA-256, normalized result, result digest, traceability digest, producing CleanroomX version and whole-snapshot SHA-256.
- Creation source limit: 16 MiB. Snapshot limit: 64 MiB.
- Verification checks integrity plus deterministic replay. Exit code 0 requires both integrity_valid and replay_consistent.
- Hashes are content identities, not digital signatures or proof of authorship/approval.

16 PROOFGRAPH EVIDENCE MODEL [CURRENT MAIN]

GUI-independent requirement-to-evidence-to-verdict traceability for the continuous compliance / digital-twin direction.

Release status ProofGraph schema v1 and its current adapters are implemented on current main after v0.102.1; they are not part of the stable release closure.

Figure 4. ProofGraph evidence and integrity chain.

Core object types

- Requirement / RequirementSet
- EvidenceSource / Evidence
- DesignEvidence / CalculationEvidence / SimulationEvidence / CommissioningEvidence / OperationalEvidence
- ProvenanceRecord / ConfidenceRecord
- ComplianceCheck / ComplianceFinding / ComplianceVerdict
- CorrectiveAction / VerificationRun / ProofGraph

Verdict vocabulary

State	Meaning
---	---
pass	Verified success with required supporting evidence.
warning	Evaluated state requiring attention without being a fail.
fail	Evaluated requirement/check failed.
unknown	Available evidence/provenance is insufficient for a verified verdict.
indeterminate	Canonical analysis evaluated, but the result interval overlaps the decision boundary.
not_checked	Known requirement/check was intentionally not evaluated, often because required evidence/mapping is absent.

Integrity gates

- Whole-graph deterministic SHA-256 can be supplied and is verified during parsing.
- Unknown fields, duplicate identifiers, unsupported kinds and non-finite values fail closed.
- Dangling and semantically inconsistent cross-references fail closed.
- Evidence provenance must be acyclic; validation uses an explicit traversal stack for deep chains.
- Provenance record IDs are globally unique across the graph.
- Explicit project_id values in one graph must agree.
- A single-finding verdict must have the same status as its finding.
- Every declared verification-run check must contribute to a verdict, and verdicts may depend only on checks declared in that run.

- PASS findings must reference evidence; when `required_evidence_kinds` are declared, PASS must reference every required kind.

Implemented adapters

Current main bridges compliance rule-pack results, pressure-design consistency, IFC/BIM semantic design evidence, ACH design evidence, airflow-balance evidence and thermal uncertainty capacity evidence into ProofGraph without creating duplicate solvers. Existing source semantics and `not_checked`/indeterminate distinctions are preserved.

Corrective actions require approval Every modeled ProofGraph `CorrectiveAction` must retain `requires_approval=true`. The evidence foundation never applies a proposed engineering change automatically.

17 BIM / IFC INTEROPERABILITY [CURRENT MAIN ENHANCEMENTS]

Semantic import, provenance, identity-preserving re-import, conflict review, resource bounds and ProofGraph evidence.

Optional dependency and CLI

```
pip install "cleanroomx[bim]"
cleanroomx-ifc import project.cleanroomx.json facility.ifc
cleanroomx-ifc plan project.cleanroomx.json facility-v2.ifc
cleanroomx-ifc reimport project.cleanroomx.json facility-v2.ifc
```

Current semantic mapping

IFC entity	CleanroomX mapping / rule
---	---
IfcSpace	Room. Explicit positive dimensions or conservative verified axis-aligned rectangular-prism geometry fallback.
IfcDoor / IfcWindow	Door / window opening.
IfcAirTerminal	Supply/return/exhaust only when explicit semantic type supports the airflow role; ambiguous types remain equipment.
Direct generic IfcFlowTerminal	Equipment; no airflow role is guessed.
IfcSensor	Sensor.
IfcFlowController / IfcUnitaryEquipment / IfcFan / IfcPump / IfcFurnishingElement	Equipment.
IfcBuildingStorey	Storey identity/name/world elevation retained; a single common storey can promote floor metadata.
CleanroomX_Space custom property set	Classification -> room classification; AnalysisRoomName -> explicit engineering room link.

Geometry fidelity

IfcOpenShell full nested placement transforms are used. Rectangular spaces at 0/90/180/270-degree plan rotations are converted exactly into the axis-aligned CleanroomX room contract. Arbitrary-angle, tilted, reflected, skewed, non-rectangular or complex solids are rejected rather than silently approximated by generic bounding boxes.

Identity and conflict-aware re-import

IFC link schema v2 binds GlobalId values to stable CleanroomX spatial IDs with source-record digests, source-derived spatial-object digests and a canonical bindings_sha256. Re-import planning classifies unchanged, source-only, local-only, converged, added, removed and conflicting changes. Different two-sided edits are explicit conflicts and block transactional apply.

Source stability and resource guards

- Native IFC source cap: 512 MiB, checked before parsing and re-enforced while hashing.
- Source is SHA-256 hashed before and after semantic extraction; source drift causes extraction to be discarded.

- Desktop review/apply re-extracts after confirmation and requires source + semantic digests to match the reviewed candidate.
- Import modifies the in-memory project; normal guarded Save remains responsible for persistence.
- IFC operations do not silently synchronize engineering analysis inputs.

ProofGraph bridge

Normalized IFC semantic fields can be emitted as design evidence bound to source filename/SHA-256, semantic SHA-256, GlobalId and field identity. Dimension evidence preserves dimension_source, distinguishing explicit IFC quantities from verified IfcOpenShell geometry fallback.

18 PERSISTENCE, RECOVERY, REVISIONS AND RUN HISTORY

How CleanroomX protects project bytes, unsaved work, prior revisions and evidence identity.

Atomic guarded project saves

Project saves serialize and validate schema-v1 data, stage the new bytes in the destination directory, flush/fsync, and atomically replace the target. A stable opened-file revision (path, size, timestamp and SHA-256) is used as an optimistic guard. External changes or deletion block Save instead of overwriting newer disk content.

Recovery autosave

- Dirty edits schedule a debounced checkpoint after about 1.5 seconds; a 60-second periodic sampler is a fallback.
- Autosave writes separate versioned cleanroomx.autosave artifacts and never writes the explicit project path.
- One background writer coalesces pending edits and suppresses identical snapshots.
- Recovery records source fingerprint, project snapshot, active raw editor draft, app version and recovery time.
- Recovery semantic comparison reparses preserved drafts through strict JSON so duplicate keys and non-finite constants remain invalid evidence.

Recovery Center behavior

Startup Recovery Center can inspect, restore as unsaved copy, or discard a selected recovery. Restore never overwrites/rebinds the original project and requires Save As. Discard deletes only the selected validated recovery artifact.

Saved revisions and project-wide Undo/Redo

Release 2 preserves validated prior project bytes before guarded overwrites and uses one bounded application-wide edit transaction history across project fields, analysis edits and spatial edits. Persistent run history and camera/view state are not rewound as design edits.

Run history

Accepted completed runs are stored as a bounded integrity-checked audit ledger tied to exact analysis input and execution provenance. Evidence freshness is distinct from design state and is checked against current input and external dependency fingerprints.

19 REPORTING, BUNDLES, BATCH AUTOMATION AND DIAGNOSTICS

Portable evidence, project handoff, deterministic execution and read-only health scanning.

Portable engineering HTML report

File -> Export Portable HTML Report creates a self-contained document containing project/analysis identity, exact submitted input SHA-256, exact input JSON, normalized result JSON, diagnostics/provenance, backend Markdown and a machine-readable cleanroomx.engineering-report payload with a deterministic SHA-256. It contains no external scripts, fonts, images or network resources.

Fresh result required HTML generation never invokes a solver. It first rechecks the completed run against the current analysis input identity; stale input means the report fails and the analysis must be rerun.

Portable project bundles

```
cleanroomx-project-bundle export project.cleanroomx.json review.cleanroomx.zip
cleanroomx-project-bundle verify review.cleanroomx.zip
cleanroomx-project-bundle extract review.cleanroomx.zip ./review
```

Bundle schema v1 packages manifest.json, project.cleanroomx.json and deduplicated dependencies. Stored ZIP members use fixed metadata and deterministic ordering so identical inputs produce identical bundle bytes.

Bundle guard	Limit / behavior
---	---
Archive size	Reject > 1,040 MiB before archive hashing.
Manifest	Reject > 1 MiB.
Bundled project	Reject > 64 MiB.
Individual dependency	Reject > 512 MiB.
Aggregate payload	Reject > 1 GiB.
Dependency count	Reject > 1,024 dependencies.
Extraction	Fully verify first; copy into private staging; publish via rename only after project loads. Never extractall().

Deterministic project batch execution

```
cleanroomx-project-run project.cleanroomx.json
```

Batch execution binds to one exact project revision, runs selected analyses in persisted project order through the shared application service, and rechecks the source revision before and after every attempt. Source drift stops new scheduling.

Project diagnostics [current main]

```
cleanroomx-project-check project.cleanroomx.json
cleanroomx-project-check project.cleanroomx.json --format markdown
cleanroomx-project-check project.cleanroomx.json --output project-diagnostics.json
```

Diagnostics are read-only. They compose spatial integrity, real analysis parsers, engineering synchronization, run-history freshness, external-dependency fingerprints and traceability hygiene. Exit codes: 0 = no errors/warnings; 1 = one or more errors/warnings; 2 = project could not be checked

safely.

20 COMMAND-LINE REFERENCE

Installed entry points on current main and when to use them.

Command	Primary use
---	---
cleanroomx	Core verify / verify-project / decay / recovery commands.
cleanroomx-gui	Desktop application, --demo, --check, --smoke.
cleanroomx-project-run	Run saved project analyses deterministically.
cleanroomx-project-bundle	Export / verify / extract portable project bundles.
cleanroomx-project-check	Read-only project diagnostics [current main].
cleanroomx-ifc	IFC import / plan / reimport [optional BIM dependency].
cleanroomx-hvac	HVAC / thermal / airflow / fan screening workflow.
cleanroomx-recovery-test	Observed particle recovery qualification.
cleanroomx-uncertainty	Room uncertainty workflow.
cleanroomx-qualification	Qualification workflow.
cleanroomx-duct-flow	Duct-network pressure-loss analysis.
cleanroomx-loop-flow	Fixed-resistance looped airflow network.
cleanroomx-pressure-network	Room pressure/leakage network [current main].
cleanroomx-loop-friction	Variable-friction loop solver.
cleanroomx-thermal-uncertainty	Bounded thermal/HVAC uncertainty.
cleanroomx-psychrometric-uncertainty	Bounded psychrometric uncertainty.
cleanroomx-fan-curve	Fan/system operating point.
cleanroomx-fan-uncertainty	Fan/system bounded uncertainty.
cleanroomx-fan-speed	Fan speed / affinity-law study.
cleanroomx-fan-duct	Fan plus duct-network composition.
cleanroomx-fan-network	Fan plus passive network.
cleanroomx-fan-loop	Fan plus looped network.
cleanroomx-fan-loop-friction	Fan plus variable-friction loop.
cleanroomx-fan-loop-friction-speed	Speed study with nonlinear variable-friction loop.
cleanroomx-fan-loop-friction-uncertainty	Uncertainty around nonlinear fan/variable-friction loop.
cleanroomx-fan-loop-uncertainty	Fan/loop bounded uncertainty.
cleanroomx-fan-loop-speed	Fan/loop speed study.
cleanroomx-damper-study	Loop resistance-multiplier scenarios.
cleanroomx-dossier	Engineering dossier composition.

cleanroomx-consistency	Cross-module consistency checks.
cleanroomx-assurance-snapshot	Create / verify design-assurance snapshots [current main].

Discover exact options Use COMMAND --help for the exact arguments supported by the installed build. This is preferable to copying an old CLI syntax from a report or earlier release.

21 WORKED SOLUTION EXAMPLES

Concrete calculations and operator recipes that correspond to implemented CleanroomX models.

Example A - ACH and particle recovery

Room: 6 m x 5 m x 3 m. Supply airflow: 2700 m³/h. Initial particle concentration: 1,000,000/m³. Target: 100,000/m³. Effective removal efficiency: 1.0.

Step 1 - volume

$$V = 6 \times 5 \times 3 = 90 \text{ m}^3$$

Step 2 - ACH

$$\text{ACH} = 2700/90 = 30 \text{ 1/h}$$

Step 3 - recovery

$$t = \ln(1000000/100000) / (30/60) = \ln(10) / 0.5 = 4.60517 \text{ minutes}$$

This matches the implemented first-order screening equation. Real observed recovery qualification should use measured samples when available.

Example B - duct section and preliminary fan power

Rectangular duct: 0.8 m x 0.45 m, airflow 5400 m³/h, length 18 m, Darcy $f=0.02$, $\rho=1.2 \text{ kg/m}^3$, local $K=1.8$. Add coil 180 Pa and other 90 Pa. Fan efficiency 0.68; motor efficiency 0.92.

$$\begin{aligned} A &= 0.3600 \text{ m}^2 \quad Q = 1.5000 \text{ m}^3/\text{s} \quad v = 4.1667 \text{ m/s} \quad D_h = 0.5760 \text{ m} \quad \text{velocity pressure} = 10.417 \text{ Pa} \\ \text{friction loss} &= 6.510 \text{ Pa} \quad \text{local loss} = 18.750 \text{ Pa} \quad \text{duct total} = 25.260 \text{ Pa} \quad \text{fan total static} = 295.260 \text{ Pa} \\ \text{air power} &= 0.4429 \text{ kW} \quad \text{shaft power} = 0.6513 \text{ kW} \quad \text{electrical input} = 0.7079 \text{ kW} \end{aligned}$$

The software keeps the canonical calculation state at full precision and rounds only presentation fields.

Example C - single-path pressure network

Process node injects 540 m³/h = 0.15 m³/s. Corridor is fixed at 0 Pa. One power-law crack has $C=0.01 \text{ m}^3/\text{s}/\text{Pa}$ and $n=1.0$. At steady state the leakage flow must equal the mechanical injection.

Solve

$$0.15 = 0.01 \cdot dP \quad dP = 15 \text{ Pa}$$

This is the same structure used in `examples/pressure_design_consistency_demo.json`; an explicit mapping can compare the +15 Pa solved difference to a +15 Pa design target.

Example D - safe project handoff

1. Save the project and make sure all current analyses you want to review have fresh completed runs.
1. Run `cleanroomx-project-check` and address errors/warnings as appropriate.
1. Export `review.cleanroomx.zip` with `cleanroomx-project-bundle export`.
1. Verify the bundle before sending it.
1. On the receiving workstation, extract to a new/empty directory and open the extracted project.
1. For a specific completed analysis, also export a self-contained portable HTML report so reviewers can inspect exact input/result/provenance without installing CleanroomX.

22 TROUBLESHOOTING AND FAILURE HANDLING

What common failures mean and the safest corrective action.

Symptom	Meaning / action
---	---
Python 3.11+ not found	Install a supported Python or create .venv in the repository. On Windows prefer start-cleanroomx.ps1.
cleanroomx-gui --check fails	Treat as application wiring/readiness failure. Fix installation/source state before normal GUI use.
GUI/Tk error	Install/enable Tk. On Linux use a real display or Xvfb for automated smoke.
Invalid JSON / duplicate key / NaN	Fix the input. CleanroomX intentionally refuses permissive JSON normalization.
Save blocked: project changed externally	Do not bypass. Use Save Project As to preserve current work, or reopen the newer disk version.
Recovery shown at startup	Inspect semantic differences. Restore as Unsaved Copy only if wanted; it will not overwrite the original.
Result disappears after edit	Expected stale-result protection. Validate and rerun the analysis.
ExternalDependencyChangedError	A referenced consistency/dossier file changed or was unstable during the run. Stabilize source files and rerun.
Fan no_intersection_in_supplied_range	The supplied fan data do not contain a crossing. CleanroomX will not extrapolate; supply valid curve data or revise system assumptions.
Pressure network non-convergence	Review connectivity, fixed boundary, path coefficients/areas, offsets, balance and solver conditions. CleanroomX does not return a false solved state.
Loop solver non-convergence	Check graph connectivity, balanced node injections, positive resistances and numerical controls.
IFC import rejected	Check source size, supported entity semantics, placement/containment, axis-aligned representability and source stability. Rejection is deliberate when fidelity cannot be preserved.
IFC re-import conflict	Use the read-only plan/review to resolve divergent local/source edits. Conflict blocks transactional apply.
Bundle verification rejected	Do not extract. Check integrity, member paths, size limits, declared dependencies and source stability.
ProofGraph parse rejected	Check strict schema fields, IDs, cross-references, provenance cycles/project IDs, PASS evidence requirements and verification-run closure.

23 SECURITY, DEPLOYMENT, MIGRATION AND ROLLBACK

Operational controls around a local engineering application.

Security posture

CleanroomX is a local Python desktop/CLI application, not a network service, authentication system, secret store, or hostile-code sandbox. Run it with normal user privileges and keep engineering source files/reports under normal OS access controls. Do not store credentials or unrelated secrets in project JSON.

Archive and file safety

Portable project bundles are treated as untrusted archives: unsafe paths, duplicate/undeclared members, encryption, unsupported compression, integrity mismatches and resource-limit violations are rejected before extraction publication.

Deployment checks

```
python -m pip install .
cleanroomx-gui --check
cleanroomx-gui --demo
```

Rollback

- Prefer Git revert of a release-changing commit/merge rather than rewriting shared history.
- Record current commit and CI run; preserve affected projects and reports; identify the last validated baseline; revert through normal review/CI.
- For migrated project formats, restore the archived original if a legacy representation is required. CleanroomX does not reverse-migrate schema-v1 to older shapes.
- Regenerate derived results using the selected validated application version rather than editing evidence files by hand.

The documented v0.102.1 functional release anchor is commit 89eaf6b1548ca0748dd72431eccc101bddec9af9; published tags should remain immutable.

24 VALIDATION EVIDENCE AND CURRENT DEVELOPMENT STATUS

What is formally recorded for the stable release and what has landed on main afterward.

Stable v0.102.1 release validation

- Final deterministic spatial-drag gate: 1008 passing tests on Python 3.11, 3.12 and 3.13.
- Focused spatial gate: 63 passed; Release 2 consolidation gate: 132 passed at PR #487.
- Windows PowerShell and CMD checkout launcher smoke: success.
- Clean wheel build/install checks across supported Python versions.
- Python 3.13 installed Tk/Xvfb GUI demo smoke: PASS.
- Legacy solver/provenance compatibility gates v0.91-v0.95 retained.

Current main at cutoff

Area	Current-main state by 30 Sep 2026
---	---
Design foundation	design_requirements, air_system_design, design_consistency, pressure_design_consistency.
Assurance	design_assurance, deterministic assurance snapshots, project diagnostics.
Pressure	first-class room pressure/leakage network plus full-precision pressure-design verdict hardening.
BIM/IFC	semantic import, source stability, identity-preserving conflict-aware re-import, explicit CleanroomX_Space semantics, storey/device orientation fidelity, 512 MiB source cap.
ProofGraph	schema v1, strict integrity semantics, pressure/IFC/ACH/airflow-balance/thermal adapters, evidence-backed PASS, run closure, provenance DAG/global IDs.
Numerical integrity	full-precision thermal, pressure, variable-friction, fan network/loop/uncertainty, HVAC/fan consistency and damper redistribution paths.
Project safety	64 MiB normal project cap and resource hardening around bundles/revisions.
Main SHA	ae80aec3f61f009597a1b01f05d67fb91b545d70 (PR #594 merged).

Open PR #595 at cutoff

Field	Verified status
---	---
Title	Use full-precision standalone fan roots in HVAC consistency
State	Open; not merged
Mergeable	Yes

Head SHA	e2fd382181b512d8ec786747db2bdc3954edb968
CI	Run #1928 - SUCCESS
Scope	Dossier-owned standalone fan/HVAC airflow consistency precision source; no equation/schema/persistence change.

Status rule Do not describe PR #595 behavior as current-main behavior until it is merged. This manual records it only as pending development.

25 ENGINEERING LIMITS, GLOSSARY AND SOURCE MAP

How to interpret outputs correctly and where each manual section is grounded.

High-value glossary

Term	Meaning in CleanroomX
---	---
Canonical calculation state	Full-precision engineering values used internally for composition and decisions before terminal formatting.
Presentation boundary	The point where values are rounded/formatted for human-facing JSON/Markdown/GUI display.
Fresh result	Completed run whose analysis kind/input identity and recorded external dependencies still match current state.
Strict JSON	JSON ingestion rejecting duplicate keys, non-finite constants and invalid schema/content.
Atomic write	Stage, flush/fsync and replace so a failed write does not intentionally truncate the last valid destination.
Provenance	Source identity/revision and derivation evidence associated with an input, calculation or result.
Content digest	SHA-256 over canonical bytes/content. Integrity identity only; not a digital signature.
not_checked	Known criterion/check was not evaluated; not equivalent to pass.
indeterminate	Evaluation completed but bounded result overlaps the decision threshold.
ProofGraph	Current-main typed evidence graph linking requirements, evidence, checks, findings, verdicts and runs.
IFC GlobalId binding	Stable mapping from imported IFC identity to CleanroomX spatial identity for auditable re-import.

Primary repository sources used for this manual

Source	Used for
---	---
README.md	Product scope, quick start, validation and release boundary.
ARCHITECTURE.md	Layering, application boundary, project lifecycle and Release 2 authorities.
docs/APPLICATION_GUI.md	Desktop installation, operator workflow, freshness, recovery, spatial workspace.
docs/LAYOUT_2D_3D.md	Canonical spatial model, controls, synchronization and geometry limits.
src/cleanroomx/calculations.py	Volume, ACH, decay and recovery equations.

src/cleanroomx/duct.py	Darcy-Weisbach/local-loss implementation and full-precision section state.
src/cleanroomx/fan.py	Fan static/power implementation and presentation boundary.
docs/AIRFLOW_FAN_MODEL.md	Airflow balance, fan duty and HVAC fan-curve duty semantics.
docs/LOOPED_NETWORK_SOLVER.md	Quadratic loop equation, solver and geometry-derived resistance.
docs/VARIABLE_FRICTION_LOOP.md	Iterated Darcy-friction loop behavior.
docs/DAMPER_STUDY.md	Resistance-multiplier scenarios and full-precision redistribution metrics.
docs/PRESSURE_NETWORK.md	Current-main room pressure/leakage equations and solver.
docs/FAN_SYSTEM_OPERATING_POINT.md	Bounded fan interpolation and quadratic system curve.
docs/THERMAL_UNCERTAINTY.md	Psychrometric corner enumeration and capacity/airflow uncertainty semantics.
docs/DESIGN_FOUNDATION.md	Current-main requirements, air-system and consistency workflows.
docs/DESIGN_ASSURANCE.md	Design-assurance matrix semantics and traceability.
docs/ASSURANCE_SNAPSHOTS.md	Snapshot schema, limits, deterministic replay and CLI.
docs/PROOFGRAPH.md	ProofGraph schema, states, integrity rules and adapters.
docs/BIM_IFC.md	IFC semantics, resource guards, re-import and ProofGraph bridge.
docs/PROJECT_BUNDLES.md	Portable bundle format, limits and extraction safety.
docs/PORTABLE_ENGINEERING_REPORT.md	Self-contained report evidence and freshness boundary.
docs/PROJECT_DIAGNOSTICS.md	Read-only diagnostics rules and exit codes.
SECURITY.md / DEPLOYMENT.md / MIGRATIONS.md / ROLLBACK.md	Operational controls and lifecycle procedures.
VALIDATION.txt / TEST_EVIDENCE.md / CHANGELOG.md	Release evidence and post-release current-main history.
pyproject.toml	Package version, Python requirement, optional dependencies and installed CLI entry points.

Repository links

Repository:

<https://github.com/3more102/CleanroomX-Cleanroom-Design-Simulation-Verification-Platform>

README: <https://github.com/3more102/CleanroomX-Cleanroom-Design-Simulation-Verification-Platform/blob/main/README.md>

Architecture: <https://github.com/3more102/CleanroomX-Cleanroom-Design-Simulation-Verification-Platform/blob/main/ARCHITECTURE.md>

Changelog: <https://github.com/3more102/CleanroomX-Cleanroom-Design-Simulation-Verification-Platform/blob/main/CHANGELOG.md>

ProofGraph: <https://github.com/3more102/CleanroomX-Cleanroom-Design-Simulation-Verification-Platform/blob/main/docs/PROOFGRAPH.md>

BIM/IFC: https://github.com/3more102/CleanroomX-Cleanroom-Design-Simulation-Verification-Platform/blob/main/docs/BIM_IFC.md

End of manual For exact current command switches and analysis input schemas, use the installed command `--help`, the repository examples, and the documentation at the same commit you are running. Preserve the commit/version alongside exported engineering evidence.